

```

In [1]: #Hashing--> CHAINING
class HashTable:
    def __init__(self,size):
        self.size=size
        self.table=[[] for _ in range(self.size)]
    def hashFunction(self,key):
        return key%self.size

    def insert(self,key):
        index=self.hashFunction(key)
        if key not in self.table[index]:
            self.table[index].append(key)
            print(f'{key} inserted at index {index}')
        else:
            print(f'Duplicate key is not allowed')

    def search(self,key):
        index=self.hashFunction(key)
        if key in self.table[index]:
            return f"{key} found at index {index}"
        else:
            return f"{key} not found"

    def delete(self,key):
        index=self.hashFunction(key)

        if key in self.table[index]:
            self.table[index].remove(key)
            return f"{key} removed successfully"
        else:
            return "Key is not found"

    def display(self):
        print("\n Hash Table")
        for i in range(self.size):
            print(f"{i}-->{self.table[i]}")

obj=HashTable(10)
obj.insert(27)
obj.insert(37)
obj.insert(47)
obj.insert(18)
obj.insert(25)
obj.insert(35)
obj.insert(15)
obj.insert(55)

# Display
obj.display()

# Search
print(obj.search(37))
print(obj.search(99))

# Delete
print(obj.delete(37))

```

```
print(obj.delete(100))
```

```
# Display Again
obj.display()
```

```
27 inserted at index 7
37 inserted at index 7
47 inserted at index 7
18 inserted at index 8
25 inserted at index 5
35 inserted at index 5
15 inserted at index 5
55 inserted at index 5
```

Hash Table

```
0-->[]
1-->[]
2-->[]
3-->[]
4-->[]
5-->[25, 35, 15, 55]
6-->[]
7-->[27, 37, 47]
8-->[18]
9-->[]
37 found at index 7
99 not found
37 removed successfully
Key is not found
```

Hash Table

```
0-->[]
1-->[]
2-->[]
3-->[]
4-->[]
5-->[25, 35, 15, 55]
6-->[]
7-->[27, 47]
8-->[18]
9-->[]
```

```
In [ ]: #Hashing-->Open addressing
class HashTable:
    def __init__(self,size):
        self.size=size
        self.table=[None]*size

    def hashfunction(self,key):
        return key%self.size

    def insert(self,key):
        index=self.hashfunction(key)
        original_index=index

        while self.table[index] is not None:
            if self.table[index]==key:
                return f"Duplicate is not allowed"

            index=(index+1)%self.size
```

```

        if index==original_index:
            return f"Hash Tabel overflow"
        self.table[index]=key
        print(f'{key} inserted at index {index}')

def search(self,key):
    index=self.hashfunction(key)
    original_index=index

    while self.table[index] is not None:#While the current position is occup
        if self.table[index]==key:
            return f'{key} found at index {index}'

        index=(index+1)%self.size

        if index==original_index:
            break
    return f"{key} not found"

def delete(self,key):
    index=self.hashfunction(key)
    original_index=index

    while self.table[index] is not None:
        if self.table[index]==key:
            self.table[index]=None

            return f"{key} deleted successfully"
            index=(index+1)%self.size

        if index==original_index:
            break

    return f"{key} not found"

def display(self):
    print("\nHash Table")

    for i in range(self.size):
        print(f"{i}-->{self.table[i]}")

obj=HashTable(10)
obj.insert(27)
obj.insert(37)
obj.insert(47)
obj.insert(18)
obj.insert(25)
obj.insert(35)
obj.insert(15)
obj.insert(55)

# Display
obj.display()

# Search
print(obj.search(37))

```

```

print(obj.search(99))

# Delete
print(obj.delete(37))
print(obj.delete(100))

# Display Again
obj.display()

```

27 inserted at index 7
 37 inserted at index 8
 47 inserted at index 9
 18 inserted at index 0
 25 inserted at index 5
 35 inserted at index 6
 15 inserted at index 1
 55 inserted at index 2

Hash Table

0-->18
 1-->15
 2-->55
 3-->None
 4-->None
 5-->25
 6-->35
 7-->27
 8-->37
 9-->47
 37 found at index 8
 99 not found
 37 deleted successfully
 100 not found

Hash Table

0-->18
 1-->15
 2-->55
 3-->None
 4-->None
 5-->25
 6-->35
 7-->27
 8-->None
 9-->47

```

In [1]: #TWO SUM
class solution:
    def TwoSum(self,arr,target):
        dict1={}

        for i in range(len(arr)):
            rem=target-arr[i]

            if rem in dict1:
                return [dict1[rem],i]
            else:
                dict1[arr[i]]=i
obj=solution()

```

```
res=obj.TwoSum([1,3,4,8,7],12)  
print(f'Result is:{res}')
```

Result is:[2, 3]

In []:

In []: